PROYECTO INTEGRADOR · MÓDULO DATA SCIENCE

FleetLogix

Optimización de operaciones logísticas

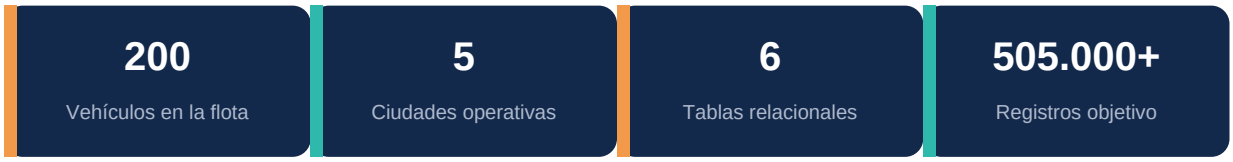
README.pdf — Guía general del proyecto

- Empresa simulada: FleetLogix (transporte y logística de última milla)
- Alcance: 200 vehículos · 5 ciudades · 6 tablas relacionales · 505.000+ registros
- Base transaccional: PostgreSQL · Data Warehouse analítico: Snowflake

1. Contexto del proyecto

FleetLogix es una empresa de transporte y logística que opera una flota de 200 vehículos, realizando entregas de última milla en 5 ciudades principales (Bogotá, Medellín, Cali, Barranquilla y Cartagena). La compañía ha operado históricamente con sistemas legacy y planillas de cálculo, y este proyecto acompaña su proceso de modernización de infraestructura de datos.

El presente documento (README.pdf) funciona como guía general del Proyecto Integrador: resume el contexto de negocio, el modelo de datos provisto, la estructura del entregable final y el estado de avance del trabajo realizado hasta el momento.



Objetivos del Proyecto Integrador

- Dominar SQL avanzado: CTEs, Window Functions y optimización de queries.
- Generar datos sintéticos masivos manteniendo integridad y realismo de negocio.
- Transformar el modelo transaccional en un Data Warehouse dimensional (Snowflake).
- Analizar y documentar una arquitectura cloud básica en AWS (RDS, S3, Lambda, DynamoDB, API Gateway).

2. Estructura del entregable

El entregable final se organiza como una carpeta de Drive con la nomenclatura **ProyectoM2_NombreEstudiante**, dividida en tres bloques: notebooks, scripts y documentación.

1. Notebooks

- **Avance_1_DataGeneration.ipynb** — generación de datos sintéticos.
- **Avance_2_SQLAnalysis.ipynb** — análisis de las 12 queries SQL.
- **Avance_3_DataWarehouse.ipynb** — modelado dimensional y ETL a Snowflake.
- **Avance_4_CloudArchitecture.ipynb** — arquitectura en la nube (AWS).

2. Scripts

- **01_data_generation.py** — genera 505.000+ registros (Avance 1).
- **02_queries_analysis.sql** — 12 consultas provistas por la cátedra (Avance 2).
- **03_optimization_indexes.sql** — índices de optimización (Avance 2).
- **04_dimensional_model.sql** — definición del Data Warehouse (Avance 3).
- **05_etl_pipeline.py** — pipeline ETL PostgreSQL → Snowflake (Avance 3).

3. Documentación

- **README.pdf** — guía general del proyecto (este documento).
- **Manual_Consultas_SQL.pdf** — explicación detallada de las 12 queries.
- **Arquitectura_Tecnica.pdf** — decisiones de arquitectura y costos AWS.
- **data_dictionary.xlsx** — diccionario de datos simple.

Contenido específico requerido: base PostgreSQL con las 6 tablas pobladas (505 k+ registros), análisis documentado de las 12 queries SQL, Data Warehouse operativo en Snowflake, pipeline ETL ejecutable con instrucciones, y arquitectura AWS documentada con diagrama técnico y análisis de decisiones.

3. Modelo de datos

El modelo relacional fue provisto por la cátedra e implementado en PostgreSQL. Está compuesto por 6 tablas: tres catálogos (vehicles, drivers, routes) y tres tablas transaccionales/dependientes (trips, deliveries, maintenance), vinculadas mediante claves foráneas.

vehicles PK

vehicle_id	SERIAL
license_plate	VARCHAR(20)
vehicle_type	VARCHAR(50)
capacity_kg	DECIMAL(10,2)
fuel_type	VARCHAR(20)
acquisition_date	DATE
status	VARCHAR(20)

drivers PK

driver_id	SERIAL
employee_code	VARCHAR(20)
first_name	VARCHAR(100)
last_name	VARCHAR(100)
license_number	VARCHAR(50)
license_expiry	DATE
phone	VARCHAR(20)
hire_date	DATE
status	VARCHAR(20)

routes PK

route_id	SERIAL
route_code	VARCHAR(20)
origin_city	VARCHAR(100)
destination_city	VARCHAR(100)
distance_km	DECIMAL(10,2)
estimated_duration_hours	DECIMAL(5,2)
toll_cost	DECIMAL(10,2)

trips PK/FK

trip_id	SERIAL
vehicle_id	INTEGER
driver_id	INTEGER
route_id	INTEGER
departure_datetime	TIMESTAMP
arrival_datetime	TIMESTAMP
fuel_consumed_liters	DECIMAL(10,2)
total_weight_kg	DECIMAL(10,2)
status	VARCHAR(20)

deliveries PK/FK

delivery_id	SERIAL
trip_id	INTEGER
tracking_number	VARCHAR(50)
customer_name	VARCHAR(200)
delivery_address	TEXT
package_weight_kg	DECIMAL(10,2)
scheduled_datetime	TIMESTAMP
delivered_datetime	TIMESTAMP
delivery_status	VARCHAR(20)
recipient_signature	BOOLEAN

maintenance PK/FK

maintenance_id	SERIAL
vehicle_id	INTEGER
maintenance_date	DATE
maintenance_type	VARCHAR(50)
description	TEXT
cost	DECIMAL(10,2)
next_maintenance_date	DATE
performed_by	VARCHAR(200)

PK = clave primaria · FK = clave foránea · UNIQUE = valor único en la tabla.

Relaciones principales

Tabla origen	Columna	Referencia
trips	vehicle_id	vehicles(vehicle_id)
trips	driver_id	drivers(driver_id)
trips	route_id	routes(route_id)
deliveries	trip_id	trips(trip_id)
maintenance	vehicle_id	vehicles(vehicle_id)

Índices básicos provistos: idx_trips_departure (trips.departure_datetime), idx_deliveries_status (deliveries.delivery_status), idx_vehicles_status (vehicles.status).

4. Generación de datos sintéticos

El script **01_data_generation.py** puebla las 6 tablas respetando integridad referencial y reglas de negocio, utilizando la librería Faker (localización es_CO) con semillas fijas para reproducibilidad (Faker.seed(42), random.seed(42), np.random.seed(42)).

Volumen de datos por tabla

Tabla	Registros	Criterio de generación
vehicles	200	4 tipos de vehículo, placas únicas, 90% activos
drivers	400	Licencias C1/C2/C3/A2, 95% activos
routes	50	Combinaciones entre las 5 ciudades principales
trips	100.000	Distribuidos según flota, conductores y rutas
deliveries	400.000	Asociadas a viajes, con estado y firma
maintenance	5.000	~1 mantenimiento cada 20 viajes por vehículo

Controles de calidad de datos

- Integridad referencial completa (sin huérfanos en trips/deliveries).
- Consistencia temporal: arrival_datetime posterior a departure_datetime.
- Peso transportado dentro de la capacidad del vehículo asignado.
- Tracking numbers únicos y no nulos en todas las entregas.
- Logging de todo el proceso en data_generation.log y resumen en generation_summary.json.

5. Cómo ejecutar el proyecto

Requisitos (Tech Stack)

- PostgreSQL 15+ y Python 3.10+
- Librerías: psycopg2, pandas, numpy, faker, tqdm, sqlalchemy
- Jupyter Notebooks y DBeaver / pgAdmin
- Cuenta trial de Snowflake (30 días) para los Avances 3 y 4
- Cuenta AWS Free Tier o AWS Academy para el Avance 4

Pasos sugeridos

1. Crear la base de datos y ejecutar fleetlogix_db_schema.sql para levantar las 6 tablas, índices y comentarios.
2. Configurar las credenciales de conexión en DB_CONFIG dentro de 01_data_generation.py.
3. Ejecutar el script de generación de datos en orden: vehicles → drivers → routes → trips → deliveries → maintenance (respeta las claves foráneas).
4. Revisar generation_summary.json y data_generation.log para validar que todas las verificaciones de calidad pasaron correctamente.
5. Continuar con el análisis SQL (Avance 2), el modelado dimensional en Snowflake (Avance 3) y la documentación de arquitectura AWS (Avance 4).

Comando de ejecución

```
python 01_data_generation.py
```

6. Estado de avance

Avance	Contenido	Estado
1 · Data Generation	Modelo analizado, datos sintéticos generados y validados (505.000+ registros)	Completo
2 · SQL Analysis	12 queries provistas (básicas, intermedias y complejas) + índices de optimización	Pendiente
3 · Data Warehouse	Modelo en estrella, ETL PostgreSQL → Snowflake y automatización diaria	Pendiente
4 · Cloud Architecture	Diagrama y documentación de arquitectura AWS (RDS, S3, Lambda, DynamoDB, API GW)	Pendiente

Este README se actualizará a medida que se completen los Avances 2, 3 y 4. Los entregables PDF adicionales (Manual_Consultas_SQL.pdf y Arquitectura_Tecnica.pdf) documentarán en profundidad las queries SQL y las decisiones de arquitectura cloud respectivamente.